Drone Security Analyst Agent

FlytBase AI Engineer Assignment: technical report

Abdul Basith Syed

1 October 2026

Built from the repository at commit `1dbbf05`, plus 22 uncommitted file changes

> This report describes what was built, how it was evaluated and what the evaluation does and does not show. Every number is taken from the repository: the evaluation snapshot in `docs/evaluation.md` (regenerated by an offline, deterministic harness and re-checked against a fresh run when this report was built), a replay of the bundled real clip and its recorded cassette, the collected test suite, and the committed scale report in `bench/`. The kind of evidence is labelled throughout:
>
> Controlled / synthetic · Real video pixels · Recorded VLM replay · Simulated telemetry, time and zone · Automated tests · Measured on one machine

Contents

1. Executive summary
2. Problem statement
3. Feature specification
4. Requirements and assignment mapping
5. Assumptions
6. System architecture
7. End-to-end data flow
8. Video and VLM processing
9. Telemetry integration
10. Frame indexing and context memory
11. Event and entity construction
12. Security and safety rules
13. Progressive / streaming processing
14. Failure handling and resilience
15. LangChain analyst architecture
16. Analyst tools and context management
17. Evidence, citations and factual-claim validation
18. Dashboard / user interface
19. Real-video experiment
20. Evaluation methodology
21. Quantitative results
22. Qualitative results and examples
23. Scalability results
24. Experimentation and ablations
25. Bonus features
26. AI-assisted development
27. Limitations and threats to validity
28. What could be improved with more time
29. Demo video references
30. Conclusion

Appendix A. Reproducing the results

1. Executive summary

What was built. A Python prototype of a drone security analyst for a fixed property. It takes scripted text-frame scenarios or real video, joins each frame to *simulated* telemetry, records what is seen as observations in a SQLite frame-by-frame index (with FTS5 search), merges repeated detections into temporal events, groups separate vehicle events into coarse "similar vehicle" entities, applies eight deterministic rules (four over camera events, four over drone telemetry) to produce evidence-linked alerts with severity ladders, recommended actions and a lifecycle, and lets an analyst answer questions and summarise the run with explicit citations to stored evidence. A scripted patrol and a video can both be processed progressively (each alert is saved the moment it fires), a read-only dashboard shows the stored evidence, and an offline, deterministic evaluation harness measures the components.

Core design. `FRAME → OBSERVATION → EVENT → MEMORY → REASONING`, with one rule: **rules decide; the LLM only explains.** Alerts, severities, events and entities come from deterministic code with configured thresholds. A language model (Gemini, through LangChain) can be used in two optional places only: as a vision model that describes frames, and as an analyst that explains stored evidence. It has no write access and cannot create, suppress or re-rate an alert, and its answers are shown only if a validator confirms that every cited id exists and was retrieved and that the facts it states in checkable forms agree with the cited evidence. A deterministic template analyst works with no key and is the fallback.

The bundled demo. The dashboard opens on a real video that is part of the repository (licensed for redistribution and for hosting in this demo): 1920×1080, 25 fps, 617 frames, 24.68 s. The frames are real video pixels; the frame descriptions are replayed from a recorded Gemini 3.6 Flash cassette, so the demo makes no model call and needs no API key and no original video URL; and the start time, the zone and the telemetry are simulated.

Headline results (Section 21 explains what each measures and where it stops): 1301 automated tests pass; on controlled fixtures the frame gate sent 5 of 40 frames (87.5% fewer calls) and detected 2 of 4 meaningful changes; all 22 labelled scenarios agree with the rules; the citation gate decided 29 of 29 adversarial cases as expected; and the replay of the real clip produced 9 observations, 33 objects, 28 events, 12 entities and 0 alerts.

> **How to read the results.** Controlled synthetic experiments (Sections 21 and 24) measure whether the code behaves as designed on authored fixtures. The real-video replay (Section 19) shows that the pipeline runs end to end on real footage and reproduces exactly; the clip is not security footage, it has no ground truth, and zero alerts is not evidence that the vision model or the security logic is accurate. Nothing here establishes real-world detection accuracy, precision or recall, and the report makes no such claim.

2. Problem statement

A docked drone patrols one property every day, but nobody watches hours of footage. The system must turn drone video and telemetry into a short, trustworthy list of things that matter (*immediate alerts*, such as a person loitering at the main gate), a *searchable history* ("show all truck events", "did the blue pickup come back?") and a *short summary* of the patrol. Two properties matter more than coverage: every alert and every answer must be traceable to the event, frame or telemetry evidence behind it, and the component that decides whether something is an alert must not be a language model that can be wrong in unpredictable ways.

The assignment asks for simulated telemetry and video frames, object and event logging with context, rule-based alerts, and a frame-by-frame indexing component, together with a feature specification, architecture and design artifacts, AI-generated components with an account of how AI was used, test cases for dynamic inputs and emergency responses, a report and demonstration videos. Section 4 maps each item to where it is met.

3. Feature specification

Value proposition. Property owners get automated monitoring, faster response to real threats, and evidence (event and frame identifiers) behind every claim, so alerts can be checked instead of trusted blindly.

Key requirements (`docs/feature_spec.md`)

#	Requirement	How it is realised
1	**Understand time, not just frames.** Consecutive frames of the same person or vehicle become one event with first-seen, last-seen and duration; repeat visits by a similar vehicle are remembered.	Temporal event aggregation and coarse entity linking (Section 11).
2	**Deterministic, evidence-backed alerts in real time.** A rule engine decides what is an alert; every alert cites its evidence, severity and a recommended action; the LLM may explain but never decides.	Eight rules (Section 12); progressive processing for a simulated patrol and for video (Section 13). "Real time" means streaming on a simulated clock.
3	**Queryable history with grounded answers.** Observations and events are indexed by time, class and zone; the analyst answers follow-ups and summarises, citing evidence ids; an answer that fails validation is replaced by a template answer.	SQLite + FTS5 (Section 10); analyst, citation gate and factual-claim check (Sections 15 to 17).

MVP scope (built). Simulated frames and telemetry to observations to a SQLite index to temporal events to coarse entity memory; eight rules with an alert lifecycle; a progressive patrol run and a progressive video path; real video through OpenCV and a real VLM behind a provider interface (with cassettes); LangChain analyst Q&A with citations and a summary; a read-only dashboard; a CLI, a pytest suite, an evaluation harness and scalability measurements.

Non-goals. Embeddings and vector databases, crowd or stationary-object rules, YOLO or multi-object tracking, multi-drone operation, real MAVLink/RTSP integration, face recognition, Docker/CI, Kafka/Redis/Kubernetes, LangGraph or multi-agent frameworks. (A read-only dashboard, battery and geofence rules and a 10K/100K benchmark were first postponed and later built.)

4. Requirements and assignment mapping

Where each assignment item is met

Assignment item	Where it is met	Report section
Process simulated telemetry and video frames	`simulate`, `ingest`, `run` (scripted frames with seeded telemetry, batch or progressive); `ingest-video` (real video, batch or `--live`)	7, 8, 9, 13
Identify objects and events and log them with context	Observations with zone, time and telemetry; events with duration and evidence; Gemini frame descriptions (recorded cassette)	8, 10, 11, 19
Alerts from predefined rules	Eight deterministic rules in `config/security_rules.yaml`; `analyze`, `alerts --show`; `run` raises them progressively	12, 13
Frame-by-frame indexing (cross-domain)	SQLite `frames` / `observations` with FTS5, queried by time, zone, object or keyword	10
Feature spec, architecture and design artifacts	`docs/feature_spec.md`, `docs/architecture.md`, the figures in this report	3, 6
VLM script, LangChain agent and context management	`vision/gemini.py`; `agent/` (bounded session memory, tools, citation and factual-claim gates)	8, 15, 16, 17
AI-generated components and AI usage	`docs/ai_usage.md`	26
Test cases, including dynamic inputs and emergency responses	`docs/testing.md`, the pytest suite, ten scripted scenarios with ground-truth labels	20, 21, 24
Sample output: logs, an alert, queryable frames	The command-line demos in the README and the dashboard	22, 18
Bonus: video summary and follow-up questions	`summarize`; `ask --follow-up` and `--interactive`; the dashboard analyst	16, 25
Detailed setup and run instructions	`README.md` (Quick start, command-line demos, dashboard, public hosting)	Appendix A
Demonstration videos	To be linked before submission	29

5. Assumptions

Explicit assumptions

Area	Assumption
Property	One fixed property, one drone, one patrol per run; six zones: `main_gate`, `driveway`, `parking_area`, `warehouse`, `restricted_zone`, `garage`.
Telemetry	Simulated: a seeded simulator (or a timeline you supply) produces position, altitude, battery and zone. A video carries none. An object's zone is the drone's zone when the frame was captured.
Time	Video time is mapped onto a simulated wall clock, so time-of-day rules (for example crossing midnight) work on any clip. "Real time" means streaming processing on this simulated clock, not a live feed.
Observations	Scripted scenarios use authored text descriptions as deterministic ground truth for tests; real video is described by a vision model behind the same provider interface (or replayed from a cassette). Model output is treated as noisy.
Identity	No face recognition and no personal identification. Vehicles are grouped only by coarse attributes (colour, body type) and always described as "similar"; people are never linked across events.
Access	The core pipeline, the tests, the dashboard and the real-video replay need no API key or network. Gemini is optional and configured through environment variables only.
Bundled clip	The clip is licensed for redistribution and hosting in this demo. It is a traffic scene, not security footage, and it has no ground-truth annotation.
Scope	Explicitly out of scope: see the non-goals in Section 3.

6. System architecture

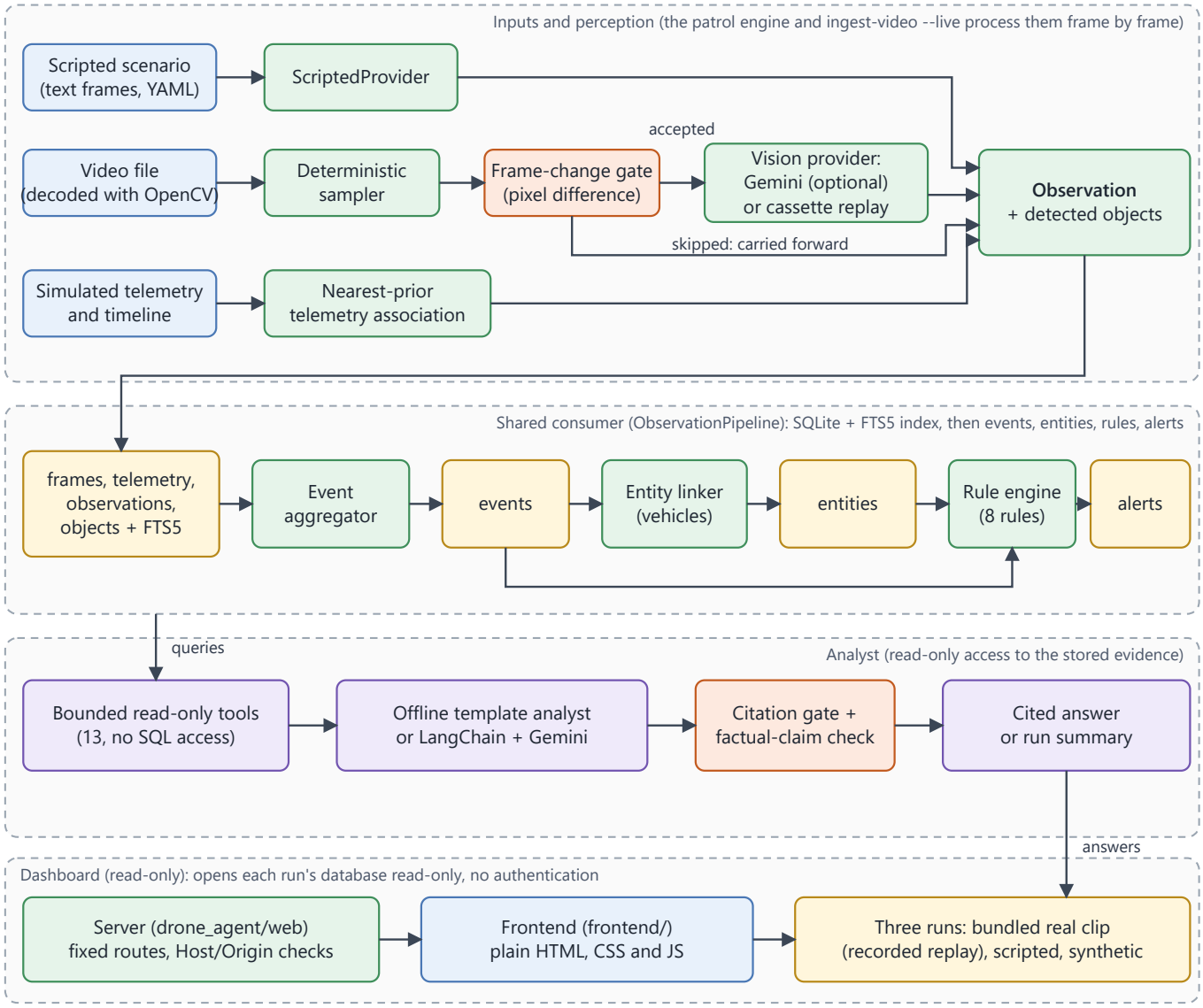


Figure 1. Current architecture. Green boxes are deterministic processing, yellow boxes are stored data, orange boxes are gates, purple boxes belong to the analyst. Gemini appears only as an optional vision provider and an optional analyst backend. The patrol engine and the progressive video driver feed the same shared consumer.

Stages and where they live

Stage	Responsibility	Package
Scenario / video	Scripted text-frame scenarios, or a real video decoded with OpenCV, sampled every N seconds and gated by a pixel-difference check.	`scenario.py`, `video/`
Vision provider	`VisionProvider.analyze(frame)` is the pipeline boundary; a real model implements the smaller `FrameDescriber.describe(frame)`. Providers: scripted, cassette (record/replay), Gemini.	`vision/`
Telemetry association	Nearest-prior sample joined to each frame; stale when older than a threshold.	`telemetry/`
Shared consumer	`ObservationPipeline`: everything downstream of perception, one observation at a time (persist, advance events and entity links, evaluate rules, persist and announce every change). Used by the patrol engine and the progressive video driver.	`runtime/consumer.py`

Stage	Responsibility	Package
Observation index	SQLite tables and an FTS5 index; every derived table links back to observations and frames.	`store/`
Event aggregation and entity linking	Repeated detections become events with duration; separate vehicle events form coarse entities.	`pipeline/`
Rule engine and alert lifecycle	Eight deterministic rules with YAML thresholds (four camera, four telemetry-safety) produce alerts with evidence, severity ladders, recommended actions and a status history.	`rules/`
Streaming runtime	`run` consumes telemetry and scripted frames tick by tick; an adaptive sampler decides which frames need a perception call; a perception agent (timeout, fallback) returns observations.	`runtime/`, `agents/`
Analyst	Thirteen read-only tools, a template analyst, an optional LangChain + Gemini loop, a citation gate and a factual-claim check.	`agent/`
Dashboard	A read-only standard-library server over each run's SQLite database and a plain HTML/CSS/JS frontend.	`web/`, `frontend/`
Evaluation harness	Offline, deterministic experiments and an ablation table; production code does not depend on it.	`eval/`

Trust boundary and dependency rules. Rules decide; the LLM explains and answers from tool output only. LangChain is imported in one file (`agent/llm.py`); the video, vision, pipeline, rules, store, model and telemetry packages never import it or the agent layer, and automated tests enforce this, so the deterministic pipeline runs without the LangChain extra. The evaluation package imports production code but no production package imports it.

---

7. End-to-end data flow

7.1 The four ways data enters

1. **Scripted, batch:** `ingest` (scenario YAML to observations and simulated telemetry in SQLite), `build-events` (observations to events and vehicle entities), `analyze` (rules to alerts). Each derived step replaces its own output in one transaction, so re-running is idempotent.
2. **Scripted, progressive:** `run --scenario NAME` consumes telemetry and frames one tick at a time through the shared consumer; alerts are stored and printed as they fire.
3. **Video, batch:** `ingest-video` samples, gates and describes frames (Gemini or a cassette) into observations; `build-events` and `analyze` derive the rest.
4. **Video, progressive:** `ingest-video --live` sends each described frame through the same shared consumer as a patrol, so observation N, its events and any alert are in the store while the video is still being processed.

7.2 A worked trace (scripted `security_events`)

1. Frames 1 to 9 are text observations of a person at the main gate (frame 5 has no detection). Each is joined to the nearest-prior simulated telemetry sample and stored with its zone, time and source.
2. The aggregator merges them into **one** event, `EVT-P-001`, 480 s long: the one undetected frame is within the missed-frame tolerance, so it does not split the event.
3. The loitering rule compares the event's 480 s with the configured 120 s threshold and 300 s critical threshold and raises `ALR-LOITER-EVT-P-001` as critical (an escalation of one alert, not two). Its reason, the compared numbers and the evidence frames (1–4, 6–9) are stored with it.
4. The analyst answers "Show me the evidence for the loitering alert" from read-only tools, citing `[alert:...]`, `[event:...]` and `[frame:N]`; every cited id is checked against the database before the answer is shown.

5. The dashboard shows the same alert, its events and frames, and the same answer, read back from the stored run.

---

8. Video and VLM processing

- **VideoSource** validates the file when it is opened: it exists and is non-empty, OpenCV opens it, the reported FPS is finite in (0, 1000], the frame count is positive, and the first frame decodes. Its identity is the SHA-256 of the file bytes.
- **Sampling** is deterministic and time-based: sample *k* is taken at frame index `floor(k × every × fps + 0.5)` (default every 2 s), so the same video and settings always select the same frames.
- **Frame gate.** The mean absolute difference between 64×36 greyscale copies of a frame and the *last accepted* frame is compared with a threshold (default 6.0, uncalibrated). The first frame is always accepted and the reference resets on a zone change. The gate is a pixel test: it is not motion detection, tracking or semantic analysis.
- **Carried observations.** By default a skipped frame is represented by a copy of the last accepted observation, labelled `<provider>+carried`, so a static scene stays one continuous activity. *A carried observation is temporal propagation of an earlier answer, not a new model inference.* It is never carried forever: after `--max-carry-seconds` (default 300 s, the adaptive sampler's bound) the provider is asked again.
- **Provider contract.** A real model returns a strict `ProviderOutput` (description; objects with type, activity, coarse attributes and optional confidence). Unknown keys, wrong types and bad ranges are errors. The pipeline, not the model, assigns zone, time and identifiers.
- **Gemini FrameDescriber** (official `google-genai` SDK). The frame is downscaled only if its longest side exceeds 1024 px, JPEG-encoded in memory (quality 85) and sent with shared instructions that tell the model to describe only visible objects and not to judge suspicion, intent, identity or severity. Native structured output is requested, then validated again by the project's own strict parser; there is no regex extraction or silent repair.
- **Confidence.** Gemini reports no calibrated probability, so none is requested. Objects are stored with a documented uncalibrated placeholder (0.5); it is not a probability and nothing downstream uses it for decisions.
- **Cassettes.** Recorded responses are keyed by `<video sha256[:16]>:<frame index>` and hold descriptions and objects only (no images, keys or absolute paths). Modes: *replay* (no key, no API call; a missing frame is an error), *record* (calls the model only for frames not yet recorded and writes the file even if a later call fails) and *live*. A cassette recorded for another video, or by another model, is refused.
- **API key handling.** `GEMINI_API_KEY` is read from the environment or a local, gitignored `.env`; it is never printed, logged or stored in the database, a cassette or a document. The dashboard and the bundled replay never read it.

> **The bundled recording.** `demo_assets/real_clip/` holds the clip (10,797,538 bytes, SHA-256 prefix `d7e3ceeba55902be`), a cassette of 8 recorded `gemini:gemini-3.6-flash` responses and a manifest that pins both files by SHA-256. The clip is licensed for redistribution and hosting in this demo, and the bundled replay needs neither the original source URL nor Gemini credentials. The dashboard refuses to build the run from a file that does not match the manifest.

> **No accuracy claim.** Frame descriptions from Gemini are used as given. How accurate they are was not measured, because no annotated ground truth exists for the real clip (Section 19). The recorded runs show that the integration works, not that the descriptions are correct.

9. Telemetry integration

All drone telemetry in this system is simulated. A video file carries none, and the assignment asks for simulated telemetry, so the prototype produces it from a seeded simulator and treats the result as if the drone had reported it.

- **Simulator.** A seeded patrol produces samples (time, latitude, longitude, altitude, zone, battery, optional signal) at a fixed interval (default 10 s) with a configurable battery drain (default 0.4% per minute), optional signal baseline and scripted dropouts; the same seed always gives the same samples. Scenarios and video timelines configure the patrol legs and the zone of each leg.
- **Video timelines.** A video is given a simulated start time and either one zone for the whole clip (`--start` and `--zone`) or a timeline file with several zones. The timeline must cover the video. The CLI labels the row "SIMULATED telemetry samples (not from the video)".
- **Association.** Each frame is joined to the nearest-prior telemetry sample. The association stores the sample's age and a `stale` flag when the age is strictly greater than 30 s. The frame inherits its zone and wall-clock time from the simulated clock.
- **Drone-safety rules** (low battery, signal loss, altitude anomaly, geofence violation) read telemetry directly, with no camera needed, and also run progressively as samples arrive (Section 13). Their alerts carry the telemetry samples that support them as evidence.
- **Labelling.** Simulated telemetry, time and zone are marked wherever they appear: in CLI output, in the analyst's run limitations, and in the dashboard (a "SIMULATED" badge next to timestamps and telemetry).

10. Frame indexing and context memory

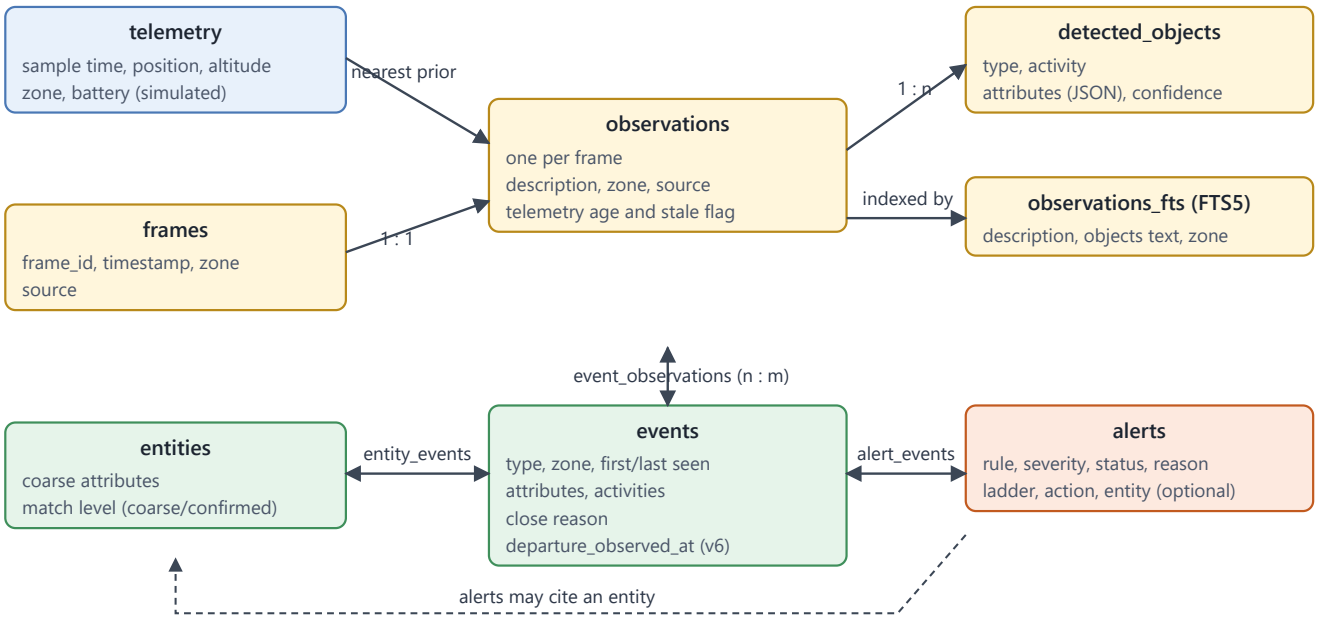


Figure 2. Stored data and evidence relationships (core tables; the current schema is version 6). Links are enforced with SQLite foreign keys.

Tables

Table	Purpose
`telemetry`	Simulated telemetry samples (time, position, altitude, zone, battery) with range checks.
`frames`	Frame-level index: frame id, timestamp, source and zone.
`observations`	One per frame: description, zone, source (`scripted`, `gemini:<model>`, `cassette`, or the same with `+carried`), and the telemetry association with its age and stale flag.
`detected_objects`	Objects in an observation: type (person, vehicle, other), activity, coarse attributes as canonical JSON, and confidence.
`observations_fts`	FTS5 index over description, an objects text and the zone name; tokenizer `porter unicode61`.
`events`, `event_observations`	Derived events (type, zone, first/last seen, attributes, activities, close reason, `departure_observed_at`) and their ordered evidence links.
`entities`, `entity_events`	Derived coarse vehicle entities and their member events.
`alerts`, `alert_events`	Rule output: rule, severity, status, reason, the numbers the rule compared, confidence, verdict, recommended action, severity ladder, triggering events and optional entity.
`alert_history`, `alert_telemetry`	The lifecycle (raised, escalated, acknowledged, resolved, with who and when) and the telemetry samples that support a drone-safety alert.
`scenarios`, `agent_runs`	Which scenario a streaming run processed, and the run's status, metrics and per-frame trace.

- **Evidence chain.** An alert links to events (and optionally an entity); an event links to observations; an observation is one frame. Any claim can be walked back to frame identifiers, which is what the analyst cites.
- **Timestamps** are stored twice: the exact ISO-8601 text with its UTC offset and a fixed-width UTC string used for ordering and range filters, because comparing ISO strings with different offsets would order them wrongly.
- **Search safety.** Search text is never passed to FTS5 as-is: each word is extracted and quoted and the words are ANDed, so input such as `'; DROP TABLE x; --` is plain words. Results are ranked with BM25 and can be combined with time, zone and object filters. FTS5 has no synonyms (a search for *truck* does not find *pickup*) and no typo tolerance.
- **Integrity.** Foreign keys are enforced; ingestion is one atomic transaction (a duplicate ingest is rejected and changes nothing); `build-events` and `analyze` replace their derived data in one transaction. Schema changes are additive and versioned with `PRAGMA user_version`; older databases are upgraded in place.
- **Why SQLite.** One file, no infrastructure, transactional, and it supports structured filters and full-text search in the same database, so a frame, its objects and its search entry are written in one transaction. Embeddings and vector search were deliberately left out of scope.

Context memory

- **Events and entities are the system's memory.** Frames are never judged in isolation: the aggregator and linker keep the temporal and cross-event context, and the rules reason over it. The incremental context engine used by the progressive paths produces the same events and entities as the batch pipeline (a test asserts it).
- **The analyst's conversational memory** is separate and small: the last six question-answer pairs per session, in memory only (Section 16).

11. Event and entity construction

11.1 Temporal event aggregation

The aggregator is a streaming algorithm over chronological observations. It is *temporal association, not object tracking*: it never looks at pixels.

- Detections of the same object type in the same zone join an open event when their stable attributes are compatible (person: `clothing`; vehicle: `color`, `body_type`; a value missing on either side is unknown). A detection with incompatible attributes starts a separate event, so two people standing together stay two events.
- An event closes on a zone change, when the next matching detection is more than 180 s later, or after more than 2 consecutive frames in its zone without a matching detection (*missed-frame tolerance*: the drone looked, the detector missed). Otherwise it extends, keeping first seen, last seen, duration and ordered evidence.
- One detection can join at most one event per frame; ties go to the most recently seen event. Event identifiers (`EVT-P-001`, `EVT-V-001`, …) follow first-seen order and are deterministic. Attributes are compared after normalisation ("dark blue" and "blue" agree); stored attributes keep their original text.

11.2 A close reason is not a departure

An event's `close_reason` (`gap`, `missed_frames`, `zone_change`, `scenario_end`) says only why observation coverage of the event ended. Whether the object actually left is recorded separately in `departure_observed_at` (schema v6): it is set only when the drone kept looking at the same zone and the object was not there. A gap, a camera move to another zone and the end of the input say nothing about departure. This matters for loitering (Section 12), whose alert resolves only at the observed departure or when an operator resolves it. Older databases are not reinterpreted: re-running `build-events` fills the new column.

11.3 Coarse vehicle entity linking

- Only *vehicle* events are linked, into entities such as `ENT-V-001`. Two events join when their normalised colour and body type are equal, they do not overlap in time, and the required attributes are present; missing evidence prevents a link and nothing is guessed. An explicit shared plate confirms an entity; conflicting plates never link.
- Entities are labelled "similar blue pickup". A coarse match is not physical identity, and no plate reading is performed. **People are never identity-linked**; there is no face recognition and no appearance model.

Known limitations (measured in Sections 19, 21 and 24): coarse matching can group different vehicles that share colour and body type; a vehicle described with inconsistent colours splits into two entities; free-text attributes from a real model fragment events; and the event model has no positions, so it cannot say which of several same-type objects in a frame is which.

---

12. Security and safety rules

Rules are deterministic functions over stored events, entities and telemetry, configured in `config/security_rules.yaml`. They decide whether an alert exists; no LLM takes part. Each alert carries a reason, the numbers the rule compared, and links to the triggering events, the entity (if any) and the frames.

The eight rules (default configuration)

Rule	Fires when	Severity
Loitering	a person event in `main_gate` lasts at least 120 s (inclusive); the same alert escalates to critical at 300 s	medium, then critical
After-hours presence	a person event overlaps the closed window 22:00-06:00 (the window crosses midnight); time of day uses the event's own UTC offset unless a property offset is configured	medium
Repeat vehicle	a coarse vehicle entity has at least 3 separate appearances; the wording is "a *similar* ... was observed in N separate appearances" and says a coarse match does not confirm the same vehicle	low
Restricted-zone entry	a person or vehicle event occurs in `restricted_zone`	high
Low battery (telemetry)	battery at or below 20%; critical at or below 10%; resolves once back at 25% or more	medium, then critical
Signal loss (telemetry)	signal below 40% is degraded, below 15% is lost, and a gap of more than 30 s in delivered telemetry counts as a lost link	medium, then high
Altitude anomaly (telemetry)	altitude outside 5-120 m; beyond 2 m or 150 m it is high	medium, then high
Geofence violation (telemetry)	the drone is more than 100 m from the property centre; critical beyond 150 m	high, then critical

- **Identifiers and duplicates.** Alert ids are deterministic, `ALR-<RULE>-<subject or episode>`, and an identical (rule, subject or episode) pair yields exactly one alert; re-running on unchanged data gives the same alerts.
- **Severity ladders and lifecycle.** A rule may raise an alert at one severity and escalate the *same* alert when the condition worsens; the alert id never changes. Each alert has a status history (open, escalated, acknowledged, resolved), a confidence, a verdict and a recommended action written for the rule and severity (a fixed table entry, not model-written). Only a rule or an operator changes a status, never a language model.
- **Resolution.** State alerts (loitering, safety) resolve when the condition ends; incident alerts (after hours, repeat vehicle, restricted zone) stay open until an operator acts. A loitering alert resolves only at the *observed* departure of the person (Section 11.2) or by an operator; the end of observation coverage does not resolve it.
- **Evidence.** `alerts --show <id>` prints the reason, the rule facts and the evidence chain down to frames (or, for drone-safety alerts, telemetry samples). `alert-action` records an operator acknowledgement or resolution in the alert's history.
- **Documented false positives and negatives** include a delivery driver waiting at the gate (loitering), authorised night staff (there is no allow-list), a person missed for longer than the tolerance splitting into events that stay under the threshold, and thresholds that were hand-set on scripted data rather than validated on real footage.

13. Progressive / streaming processing

- **One consumer for every producer.** `ObservationPipeline` is the single implementation of what happens to an observation once it exists. Both the scripted patrol engine (`run`) and the progressive video driver (`ingest-video --live`) hand their observations to it, so there is no video-specific event, entity or rule code. Telemetry joins the same way: a sample is stored, then the drone-safety rules run.
- **Progressive commits.** Each step is committed and announced before the next frame is decoded and described, so observation N, the event it extends and any alert it raises are in the store while the input is still being processed. Ending the input closes open events as `scenario_end` and takes a last look with every rule.
- **Adaptive sampler (patrol engine).** A frame is analysed on a scene change, a zone change, a telemetry change, an open event that has not been looked at for 120 s, or after 300 s without a call; otherwise the last observation is carried forward and labelled as carried. For scripted frames the scene signal is a multiset of (class, activity, attributes), so an activity-only change or an extra identical-looking object counts as a change.

- **Pacing and views.** `--speed` sets simulated seconds per wall second (0 means as fast as possible); the live terminal view prints the alert lifecycle as it happens.
- **Batch equivalence.** With sampling switched off the final database of a progressive scripted run equals the batch pipeline's, and the progressive and batch video paths reach the same final events, entities and alerts; tests assert both.
- **What "real time" means here.** It describes *when* things are computed on a simulated clock, not how good a description is and not a live feed; no live drone or RTSP stream was used.

---

14. Failure handling and resilience

What happens when something fails

Failure	Behaviour
Unusable video file	The source is validated when opened (missing, empty, garbage or truncated files, a bad FPS, a first frame that does not decode); a clear error is raised before anything is stored.
Gemini errors	Missing key, authentication failure, unknown model and rejected requests fail immediately (no fallback model). Rate-limit, timeout, network and server errors get at most two retries (2 s and 5 s); malformed output gets exactly one retry; the request timeout is 60 s. Error text is scrubbed of the key and the raw SDK exception is not chained.
Malformed model output	Output must pass the strict `ProviderOutput` contract; unknown keys, wrong types and bad ranges are errors, never silently repaired.
Cassette miss or mismatch	A missing frame in replay mode is a clear `CassetteMiss` (the inner provider is never called); a cassette recorded for another video or model is refused up front. Record mode writes the file even if a later call fails, so a rerun resumes.
Provider failure on one frame	In the patrol engine the perception agent tries providers in order with a timeout per attempt; a frame that every provider fails on is dropped and counted, never invented, and telemetry safety checks carry on. In `ingest-video --live` a failed call (error, malformed output, replay miss, or `--call-timeout` exceeded) gets an explicit failed outcome with a short classified reason (`malformed_output`, `replay_miss`, `timeout`, `provider_error`), the run continues, and `--fail-fast` stops at the first failure with what was already committed kept.
Stale carried descriptions	A carried observation is bounded by `--max-carry-seconds` (default 300 s); after that the provider is asked again.
Interrupted derived builds	`ingest` is atomic and rejects duplicates; `build-events` and `analyze` replace derived data in one transaction, so a failure leaves the previous data intact.
Analyst model failure	An unavailable model (authentication, quota, network, timeout) degrades to the deterministic template answer, labelled as such; a failed validation gets one correction turn, then the same fallback.
Altered demo assets	The dashboard checks the bundled clip and cassette against their SHA-256 manifest and refuses to build the run from a file that differs.
Dashboard errors	A panel whose request fails shows its error with a Retry button while the rest of the page keeps working; a missing frame image shows a note instead of a broken image; stale responses from a run the user has left are ignored.
Hostile or wrong requests	The server accepts only fixed routes, checks Host and Origin, caps body and query size, and sends a strict Content-Security-Policy; public mode accepts exactly one configured hostname (Section 18).

15. LangChain analyst architecture

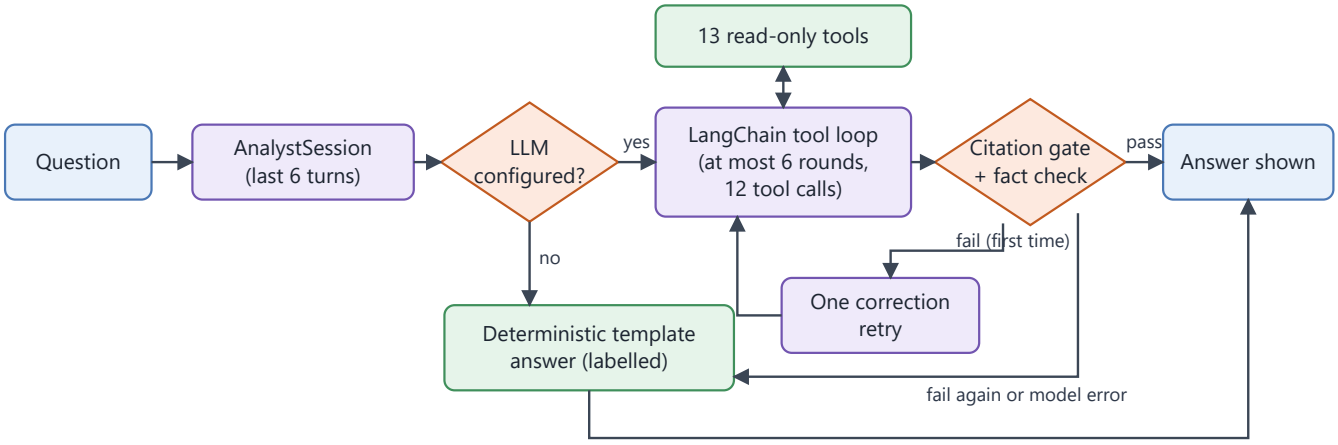


Figure 3. Analyst path. The template analyst is both the offline mode and the fallback; nothing is shown that has not passed the gates or come from the deterministic template.

- **Boundary.** The analyst may explain and summarise stored evidence. It cannot create, suppress or re-rate alerts, change event or entity links, or add observations: it has no write path (the tools are read-only methods, there is no SQL tool).
- **Why LangChain is confined.** It is needed only to drive a tool-calling chat model. `agent/llm.py` is the only module that imports it; tools, citations, the session, the template analyst and the summary are plain Python, so the deterministic pipeline and the offline analyst run without the `[agent]` extra (`langchain-core`, `langchain-google-genai`; no LangGraph). Tests enforce the import rule.
- **Modes.** `ask` and `summarize` take `--provider offline|gemini|planner|replay`. *offline* is the deterministic template analyst (no key); *gemini* is the LangChain + Gemini tool loop; *planner* is a scripted tool-calling stand-in (no LLM) that drives the real tool loop, caps and gates offline; *replay* replays recorded analyst turns from a cassette. The dashboard uses the offline template analyst.
- **The tool loop.** With an LLM configured, a LangChain chat model (Gemini) runs a hand-written tool-calling loop bounded to 6 model rounds and 12 tool calls per answer. LangChain provides only the model and tool binding; there is no agent framework. The analyst model is configured separately from the vision model (`ANALYST_MODEL`).
- **System prompt.** It requires: facts only from tools, inline citations, observation kept apart from inference, "similar" vehicle wording, no claim that the same person appears in separate events, no identity or intent claims, alerts reported exactly as stored, and "Insufficient evidence" when the evidence does not support an answer. A test checks that every tool a prompt names is registered.
- **Template (offline) analyst.** Deterministic, pattern-matched and not language understanding: alerts by rule or id, events, entities, zone summaries, blue-pickup-style vehicle questions, after-hours people, objects, the run summary and a labelled keyword search; everything else gets "Insufficient evidence" plus the list of supported questions. Output is labelled "template answer (deterministic, no LLM)".

16. Analyst tools and context management

- **Thirteen read-only, bounded tools:** `query_events`, `search_events`, `search_frames`, `get_event`, `get_frame`, `get_alert`, `get_evidence`, `entity_history`, `list_entities`, `list_alerts`, `telemetry_at`, `get_timeline`, `summarize_run`. One name per tool (the run summary has no alias); invalid arguments return an error object instead of raising, so a model can correct itself. A few older methods remain only for the template analyst and are not offered to a model.
- **Bounds.** Every result is capped (default 8 and at most 15 observations per search; 20 events, alerts or entities; 15 evidence observations per event; frame ids listed per event or alert capped; descriptions cut to 200 characters)

and reports `total`, `returned` and `truncated`; a tool result over 12,000 characters is refused.

- **Sessions.** A session keeps the last six question-answer pairs in memory (no persistence, no multi-user state) and passes them to the LLM. Template follow-ups such as "Where was it seen?" resolve a pronoun only when the previous answer named exactly one entity, alert or event; otherwise the answer is "Insufficient evidence" rather than a guess. `ask --follow-up` and `--interactive` drive a session from the command line; the dashboard keeps one server-side session per run.
- **Summaries.** `summarize` renders bounded structured facts (counts, time span, zones, object counts, events with their alerts, entities, and data-driven limitations such as "telemetry is simulated" and "N observations are carried forward"); with an LLM the same facts may be rewritten as prose under the same gates. A follow-up is answered from stored evidence, never by reprocessing the video.
- **Truncation is never silent.** Running the analyst on the first real-video recording showed that the template analyst silently dropped results (3 of 11 vehicle entities and 20 of 25 events). It now states what was cut ("showing 20 of 25 events, earliest first; 5 not shown"), lists the remaining entities briefly, and adds nothing when nothing was cut; answers that were not truncated were verified byte-for-byte unchanged.

---

17. Evidence, citations and factual-claim validation

Answers cite internal references with tokens such as `[event:EVT-V-001]`, `[alert:ALR-REPEAT-ENT-V-001]`, `[entity:ENT-V-001]`, `[obs:obs-10]` and `[frame:10]`. An LLM answer is shown only if all of the following hold:

1. every cited id exists in the database and was returned by a tool in this conversation;
2. the answer cites something, unless it says "Insufficient evidence"; a refusal that follows a failed tool call is not accepted as "no evidence";
3. it contains no malformed token and no citation-shaped token of an unknown type (for example `[evt:...]`);
4. it contains no identity or intent wording from a short phrase list ("the same person", "burglar", "criminal", ...), unless a negation cue stands in the same clause within eight words before it;
5. **the facts it states in checkable forms agree with the evidence cited in the same sentence** (the factual-claim check below).

On failure the model gets one correction turn naming the problems; if the second answer also fails, or the model call fails, the deterministic template answer is returned instead, labelled as such. Each LLM-backed answer reports call metrics outside the answer text (model, rounds, tool calls, correction attempts, fallback yes or no, latency).

The factual-claim check

A citation proves that the evidence exists, not that the sentence around it is true: "The pickup was at the main gate [event:EVT-V-001]" cites a real event that happened in the parking area. `agent/factcheck.py` closes part of that gap deterministically. It reads claims out of an answer's sentences with fixed, conservative patterns, anchors each to the evidence cited in the same sentence, and classifies it as *supported*, *contradicted* (the evidence has that fact and it differs) or *unsupported* (the cited evidence carries no such fact); only the last two fail validation.

- **Checked:** alert severity, status and rule; zones; person and vehicle types; activities; clock times; durations; appearance counts; totals of alerts, events, observations and entities (with the filters the sentence itself states); object counts in a cited frame. Recommendations are not facts, but the premises inside them are checked.
- **Not checked, by design:** anything not stated in one of those forms. This is not natural-language understanding or entailment, and no language model takes part; a claim phrased in a way the patterns do not recognise is simply not seen.
- **How it was hardened.** Probing generated answers exposed false positives, all fixed before acceptance: query-window times, a clock time such as "12:03" read as the count 3, zone-scoped counts compared with store totals, observation counts on event lines, the closed-hours window and UTC offset read as times, double counting of `[obs:]` and `[frame:]`, and ambiguous words such as "sat" and "no". The citation validator itself was not weakened.

- **Measured boundary** (Section 21): on the authored probe the citation check alone accepted all 22 unsupported answers; with the factual-claim check the production gate rejects 13 and still accepts 9.

18. Dashboard / user interface

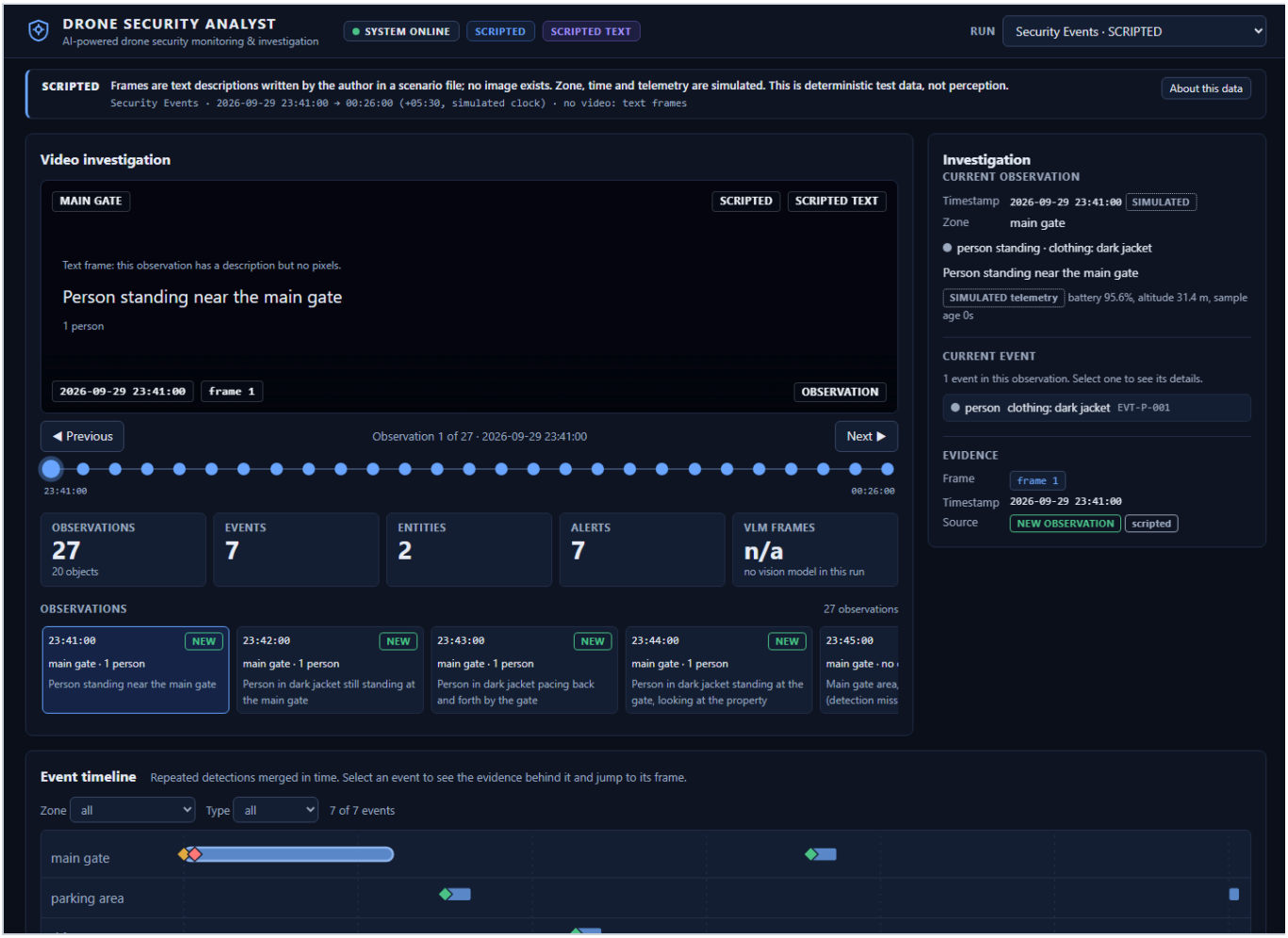


Figure 4. The dashboard on the scripted run (this run has no video pixels): header with status chips and run selector, the data-kind strip, the viewer with its overlays and observation timeline, run totals, the observation strip, the investigation sidebar and the start of the event timeline. The default run in the demo is the bundled real clip.

- **What it is.** A read-only evaluator UI: a standard-library Python server (`drone_agent/web/`) that opens each run's SQLite database read-only and serves plain HTML, CSS and JavaScript (`frontend/`, three files and an icon). No React, no Node, no build step and no CDN. It runs offline with no model call and no API key.
- **Three runs.** `--demo` builds them with the existing CLI commands and opens on the first: the **bundled real clip** (real frames with recorded VLM replay), the **scripted** `security_events` scenario (authored text frames, with alerts) and a **synthetic** video (real pixels of shapes, canned descriptions, not a vision model).
- **The page.** A viewer with overlays (zone, run kind, current and total video time, frame number, EVENT DETECTED or OBSERVATION, CARRIED FORWARD); an observation timeline and strip (each marked NEW or CARRIED); an investigation sidebar (current observation, current event, evidence); run totals; an event timeline with stacked rows and alert diamonds; an alert center (a run with no alerts shows "0 ALERTS" as a result, not an error); entity memory; the analyst with ANSWER and EVIDENCE / CITATIONS shown separately; and the seven-stage pipeline.
- **Honest labelling.** The kind label and the one-sentence description of the data are always visible; the replay chip follows each observation's declared provenance (a synthetic demo is never called a vision model); zone, time and telemetry are marked as simulated; every number is read from the API, and a test compares what the page shows with what the API returns for all three runs.

- **Safety properties.** Bound to localhost by default; fixed static-file whitelist; Host and Origin checks; body and query caps; strict Content-Security-Policy (no inline scripts or styles, no external resources); databases opened with SQLite `mode=ro`; a wording guard in front of the analyst for identity, intent and criminality questions (a wording-based filter, not a semantic classifier).
- **Public hosting is opt-in and has no authentication.** `--host 0.0.0.0` is accepted only together with `--public-host HOSTNAME`; the server then accepts exactly that hostname and its https origin and refuses every other Host or Origin. It serves read-only demo data, needs no key, and this report claims no deployed URL.
- **Accessibility and robustness.** One dark theme with colours checked for WCAG AA contrast by a test; real buttons with visible focus rings and keyboard operation; reduced motion respected; a layout that adapts from desktop to phone; per-panel loading, empty and error states.
- **Tested in a browser.** 87 UI tests: static checks plus a real headless Chrome or Edge that verifies the page against the API, the interactions, run switching, the analyst, error states, keyboard use, and no console errors, broken images or horizontal overflow at five viewport sizes (the 39 browser tests are skipped where no browser is installed).

19. Real-video experiment

Real video pixels · Recorded VLM replay · Simulated telemetry, time, zone

> **Scope.** One real clip was run through the whole pipeline with real Gemini output, which closes the gap that scripted detections do not exercise a real vision model. **The clip is a high-angle traffic-intersection video, not security footage**, and there is **no ground truth**. The result shows that the pipeline runs end to end on real footage and reproduces exactly; it does not measure perception accuracy, detection performance, precision or recall.

The clip and the replay settings (all verified from the bundled files)

Item	Value
Licence	Licensed for redistribution and hosting in this demo; the bundled replay needs neither the original source URL nor Gemini credentials.
File	`demo_assets/real_clip/clip.mp4`, 10,797,538 bytes, SHA-256 prefix `d7e3ceeba55902be`
Video	1920×1080, 25 fps, 617 frames, 24.68 s; a street intersection with traffic and pedestrians
Recorded answers	a `gemini:gemini-3.6-flash` cassette with 8 validated responses (descriptions and objects only), keyed by video hash prefix and frame index
Sampling and gate	one frame every 3 s (9 candidates, frame indexes 0, 75, ..., 600); the gate (threshold 6.0) accepts 8 and carries 1 forward (frame 525)
Telemetry, time, zone	simulated: start `2026-09-29T14:00:00+05:30` and zone `main_gate` supplied by the replay settings and applied to the whole clip
Confidence	uncalibrated placeholder 0.5 for every object

Replay result (re-derived by replaying the bundled clip and cassette when this report was built; no model call)

Stage	Result
Observations	9 (8 replayed from the cassette, 1 carried forward)
Objects	33 (17 person, 16 vehicle)
Events	28 (15 person, 13 vehicle); 24 of them span a single frame; the longest lasts 6 s
Entities	12 coarse “similar” vehicle groups, at most two appearances each
Alerts	0

19.1 Why there are zero alerts

None of the rules fired, for reasons that follow from the settings and not from what the footage shows. Loitering needs a person event of at least 120 s in `main_gate` (the clip lasts 24.68 s and the longest event is 6 s); after-hours presence does not apply because the simulated start is 14:00 and the closed hours are 22:00 to 06:00; the simulated zone is not restricted; repeat vehicle needs three separate appearances (the maximum here is two); and no telemetry-safety rule fired on the simulated timeline. Zero alerts is **not** a security-detection result, not a specificity measurement and not evidence that the vision model or the security logic is accurate; any alert on this clip would equally be a consequence of the simulated time and zone.

19.2 Two recordings, and what they show

The repository documents two recordings of the same clip. The *first* recording (written up in `docs/real_video_experiment.md`) gives 25 events and 11 entities when replayed on the final code; the *bundled* recording gives 28 events and 12 entities. Both give 9 observations (8 replayed, 1 carried), 33 objects and 0 alerts. The two recordings differ in the wording and attributes the model returned for the same frames, so the deterministic stages built different numbers of events and entities from them: the pipeline is the same and the input differs. The recording statistics of the live runs (retries, latency) are not stored in the repository and are not quoted here.

19.3 Findings (documented for the first recording; the same effects appear in the bundled one)

1. **Event fragmentation from free-text attributes.** Person events require the `clothing` text to match after lower-casing. The model's wording varies ("dark clothing", "dark coat", "dark jacket and jeans", …), so most detections start their own event; a frame can hold several people and vehicles and observations carry no position, so which object is which cannot be inferred. In the first recording 20 of 25 events are single-frame; in the bundled one 24 of 28.
2. **Inconsistent VLM attributes.** Colour and body type are not stable descriptions of the same vehicle across samples (for example a white hatchback, a blue hatchback and a black hatchback in three different samples).
3. **Coarse linking limitation.** The multi-appearance entity links a "white van" at frame 0 with a white van 15 s later; whether it is one vehicle cannot be determined from this data, which is why the system calls it a coarse, unconfirmed match.
4. **Carried observation.** Frame 525's objects are a copy of frame 450's, so event durations that extend over it include propagation, not a new detection.
5. **No ground truth.** The model's descriptions were never compared with a human annotation.

---

20. Evaluation methodology

The evaluation harness (`python -m drone_agent evaluate`) is offline and deterministic: two full runs give identical functional results, it makes no model or network call, and every table in Sections 21 and 24 was generated by executable code and cross-checked against a fresh run when this report was built (the build fails if they differ).

- **Baseline is the production default.** Ablations change one setting inside the harness (for example `entities=[]`, a different `max_missed_frames`, gate threshold 0); production code is untouched.
- **No tuning against the fixtures.** Thresholds were not adjusted to make results look better; the gate thresholds swept were chosen from the observed score distribution and the default stayed 6.0. Evaluation fixtures were not changed to improve any score, and negative results were kept.
- **Scripted detections do not measure perception.** Fixtures are authored or generated, so results say whether the code behaves as designed, not how well a vision model sees. Labels (scenario `expect:` blocks) were written by the same author who wrote the rules.

- **The kind of evidence is labelled** on every result: Controlled / synthetic, real video with recorded replay, automated tests or one-machine timing.

The experiments and what each one measures

Experiment	Question	Method
Frame gate	Model calls avoided, and what the gate misses	40 generated frames with exact ground-truth transitions (large, medium and small object changes, illumination-only steps), the real `FrameGate` at 8 thresholds
Compression	How much repeated evidence collapses into events	production aggregator on scripted scenarios and a controlled 60-frame dwell; evidence-retention invariants
Flicker	Robustness to missed detections	`max_missed_frames` 0 (ablated) versus the default; a 0-4 missed-frame gap sweep
Entity memory	What linking adds	events with coarse linking versus `entities=[]`; seven vehicle-linking controls
Rules	Rule correctness without an LLM	22 labelled scenarios × the 4 camera rules; count-based TP/FP/TN/FN
Ground truth	Streaming alerts against author-written labels	10 scenario files with `expect:` blocks run headless; alert precision, recall and F1 (2TP / (2TP + FP + FN)), severity ladder, timing and detection latency; each scenario is label-satisfied or exact
Citation gate	Does the gate reject what it should	29 adversarial analyst outputs through the real analyst session (scripted model boundary)
Faithfulness probe	Which unsupported answers still pass	answers with valid citations but unsupported claims, through the citation check alone and through the production gate
Adaptive sampling	Calls saved and changes missed	generated 600-frame streams with known meaningful changes, against a pixel gate and a fixed-refresh baseline
Scale	How cost grows with observations	generated observations, per-stage median timings (one machine; Section 23)

20.1 The automated test suite Automated tests

No test needs a Gemini key or a network connection: the Gemini SDK is replaced by a fake client, the analyst model by a scripted stand-in or a fake chat model, and a developer's real `.env` is never read. The shipped scenarios pin known facts (`security_events`: 7 events, 2 vehicle entities, 7 alerts; `normal_day`: 0), the bundled clip replays to a fixed result, and offline analyst answers recorded before the truncation fix must stay byte-identical when nothing is truncated. Mutation checks (deliberately breaking the code to see whether the tests notice) were used during development. Live Gemini behaviour, perception accuracy, field performance and the semantic truth of analyst prose are not covered by tests.

Test organisation, counted from the collected suite (1301 tests in 53 modules)

Area	Modules	Tests	What is verified
Domain and simulation	6	70	strict immutable models, timezone-aware time, seeded determinism, midnight crossing, dropouts and the stale-telemetry boundary, scenario validation
Storage and indexing	4	74	schema and foreign keys, exact round trips, timezone-safe ordering, filters, FTS5 ranking and safe search text, atomic ingest, in-place migrations, CLI queries
Events, entities, observed departure	7	106	evidence and duration, missed-frame tolerance, gap and zone splits, incremental equals batch, a close reason is not a departure, coarse linking, plates, people never linked, idempotent rebuilds
Rules, safety and lifecycle	4	113	inclusive thresholds, after-hours window edges, repeat vehicle, restricted zone, battery, signal, altitude, geofence, escalation of the same alert, operator actions, illegal transitions refused
Progressive runtime	4	69	tick ordering and pacing, alerts persisted mid-run, streaming equals batch, sampling reasons, perception timeout and fallback, the run command
Video, providers, resilience	7	185	unusable files, sampling, gate, carry bound, cassette record/replay/miss/mismatch, strict provider output, Gemini key handling and retries (SDK mocked), progressive equals batch, classified failures, timeouts
Analyst and grounding	6	265	bounded read-only tools, citation validation, the factual-claim check, retry then fallback, sessions, one tool-name contract, LangChain adapter with a fake chat model, truncation notices
Evaluation harness	5	124	determinism, metric maths and zero denominators, label-satisfied versus exact, one-to-one matching, experiment mechanics, no network or key
Dashboard server, hosting, real demo	6	199	read-only API, localhost guard, static bundle, frame images, the question guard, the opt-in public hostname, the bundled clip and cassette checked against their manifest and replayed with no key and no network
Dashboard UI	2	87	required structure, no inline styles, WCAG AA contrast, no hardcoded results; a real headless browser compares the page with the API and checks interactions, errors, keyboard use and five viewport sizes
Repository rules	2	9	core packages never import LangChain, repository layout

21. Quantitative results

Each row states the result, what the metric measures and where it stops. The first rows are controlled / synthetic or test results; the real-video rows are an integration replay.

Verified results and their limits

Metric	Result	What it measures	Limitation
Test suite	**1301 tests pass** (offline; 39 browser tests skip without Chrome or Edge)	that the code behaves as specified: boundaries, determinism, error paths, security checks	tests pin designed behaviour; they do not measure perception accuracy or field performance
Frame gate	**5 of 40** frames sent (87.5% fewer calls); **2 of 4** meaningful changes detected	model calls avoided versus meaningful changes missed, on controlled frames at the default threshold 6.0	one synthetic fixture with a clean background; the gate is pixel-based; both illumination-only changes were accepted (2 irrelevant frames sent); the default is uncalibrated and was not tuned
Flicker tolerance	with tolerance: **1 event + 1 alert**; without: **2 events + 2 alerts**	whether one undetected frame splits a loitering episode (the fragments each alert at the warning level instead of one escalated alert)	one scripted scenario; the tolerance is bounded, a longer gap still splits

Metric	Result	What it measures	Limitation
Entity memory	**1** repeat-vehicle alert with linking, **0** without; **1** false link in 7 controls	what coarse linking contributes to the repeat rule and to "seen again" questions	the controls check the documented contract, not a false-link rate; a coarse match is never identity
Rules	**22 of 22** labelled scenarios agree in alert counts	agreement of the four camera rules with expected alert counts per scenario	labels and rules share an author and detections are scripted; this is count agreement, not real-world precision or recall
Ground truth	**10 of 10** scenarios satisfy their labels; **9 of 10** match exactly (18 true positives over 18 expected alerts)	whether a streaming run reproduces the expected events, alerts, ladders and timing; "exact" also requires nothing produced beyond the labels	author-written labels, perfect scripted perception; event and entity labels are subset labels unless declared exhaustive
Citation gate	**29 of 29** adversarial cases decided as expected	decision accuracy on authored adversarial analyst outputs (19 expected rejects, 10 expected accepts)	the validator was corrected after this suite exposed two wrong decisions, so the figure is not independent evidence of general reliability
Faithfulness probe	citation check alone accepts **22 of 22** unsupported answers; production gate rejects **13** and accepts **9**	the failure boundary of citation validation, and what the factual-claim check adds	the 13 expressible cases were authored to fit the checker's classes, so catching them is true by design; not hallucination-detection accuracy
`security_events`	**27** observations, **20** detections, **7** events (ratio 2.86), 2 vehicle entities, **7** alerts	the headline scripted scenario end to end	scripted; the compression ratio is descriptive, not an accuracy measure
`normal_day`	**0** alerts	that a benign day stays silent	one control scenario (`benign_delivery` is another in the ground-truth set)
Real-video replay replay	**9** observations (8 replayed, 1 carried), **33** objects, **28** events, **12** entities, **0** alerts	that the pipeline runs end to end on real footage and reproduces exactly from a recorded cassette	not security footage; **no ground truth**; telemetry, time and zone are simulated; the descriptions are replayed, not live; zero alerts says nothing about detection accuracy
Real clip pixels	1920×1080, 25 fps, 617 frames, 24.68 s, 10,797,538 bytes; `gemini:gemini-3.6-flash` cassette	the identity of the input	one short clip, one model, one recording of it
Scale one machine	see Section 23	per-stage and query cost against generated observation counts	previously measured, single-machine, synthetic data; not a fresh benchmark and not a universal claim

22. Qualitative results and examples

22.1 An evidence-backed alert Scripted

`alerts --show ALR-LOITER-EVT-P-001` on `security_events` prints a critical alert whose reason reads: *"Person stayed in main_gate for 480s, reaching the 120s loitering threshold (event EVT-P-001). It passed the 300s critical threshold and was escalated."* Its rule facts list the comparison (`duration_seconds >= threshold_seconds`), the durations (480 s against 120 s and 300 s) and the zone, and its evidence chain runs through event `EVT-P-001` to frames 1–4 and 6–9 (frame 5 had no detection and is tolerated). Every number comes from the stored run.

22.2 A grounded analyst answer Scripted

Asked "Why was the repeat-vehicle alert raised?", the offline analyst answers (labelled *template answer (deterministic, no LLM)*) with `[alert:ALR-REPEAT-ENT-V-001]`: a low-severity alert assigned by the deterministic rule, *"A similar blue pickup was observed in 3 separate appearances (events EVT-V-001, EVT-V-002, EVT-V-003). This is a coarse attribute match; it does not confirm the same vehicle."* It then lists the compared facts (appearance count 3 against a minimum of 3, match level `coarse_match`, zones driveway, main gate and parking area), the status and the recommended action, noting that the action is a fixed table entry and not model-written.

22.3 What the production gate still accepts Controlled

These 9 unsupported answers, whose citations are all valid, are still accepted by the production gate because their wording is outside the claim forms it reads:

- `capability_overclaim`: persistent tracking
- `capability_overclaim`: vehicle tracking wording
- `identity_evasion`: very same pickup
- `identity_evasion`: identical truck
- `identity_evasion`: that same guy
- `identity_evasion`: person from earlier
- `intent_evasion`: casing the property
- `intent_evasion`: planning a break-in
- `boundary_overclaim`: analyst claims it made the alert

and it rejects the 13 whose claims are in checkable forms:

- `wrong_zone`: 2
- `wrong_duration`: 2
- `wrong_severity`: 2
- `wrong_rule`: 1
- `wrong_count`: 2
- `wrong_type`: 1
- `wrong_activity`: 2
- `wrong_time`: 1

22.4 Fragmentation on real output Recorded replay

On the real clip the model describes the same kind of person as "dark clothing", "dark coat" and "dark jacket and jeans" in different frames, and the aggregator, which requires the clothing text to match, starts a new event each time: 24 of the 28 events in the bundled replay span a single frame. The scripted `security_events` run, with consistent authored text, compresses 20 detections into 7 events (ratio 2.86). The difference is a property of free-text VLM output, and a limit of the event model, not a quality score for either input.

22.5 A false link the system admits to Controlled

The entity-memory controls include two genuinely different pickups with the same coarse attributes; coarse linking groups them. That is the documented limitation of coarse matching, and the reason every label says "similar" and no wording claims identity.

22.6 Truncation that says so Recorded replay

Before the fix, asking the offline analyst about the first real-video recording listed 20 of 25 events with no note. Now it states "showing 20 of 25 events, earliest first; 5 not shown", so a reader can see that the list is partial.

23. Scalability results Previously measured, one machine

> **These are not a fresh benchmark.** Every number in this section was measured earlier on a single development machine and is quoted from the repository (the scale section of `docs/evaluation.md` and `bench/scale_report.md`); this report did not re-run them. They use generated observations, exclude vision-model latency, and make no universal performance claim.

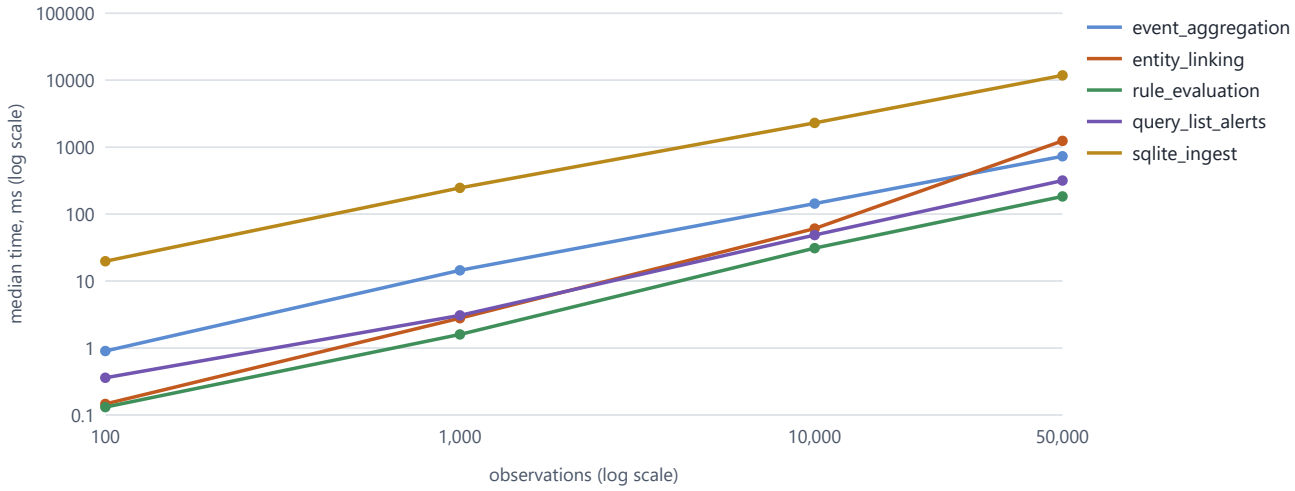


Figure 5. Median time by number of generated observations (log-log; a straight line of slope 1 would be linear scaling). Previously measured on one machine.

Median time in milliseconds by number of generated observations (SQLite ingest and derived-table writes are single runs)

stage	100	1,000	10,000	50,000
event_aggregation	0.901	14.483	143.559	732.383
entity_linking	0.146	2.788	60.701	1242.49
rule_evaluation	0.131	1.598	31.04	183.465
query_fts_keyword_top10	0.097	0.724	1.254	4.144
query_filter_zone_type_window	0.818	1.551	2.108	1.538
query_count_matching_zone	0.088	0.413	2.572	22.169
query_list_events_by_zone	0.282	1.731	15.832	110.172
query_list_alerts	0.359	3.075	48.709	317.248
sqlite_ingest	19.782	246.523	2298.01	11790.6
sqlite_persist_derived	15.743	36.001	253.199	914.648

What the evaluation harness's earlier run shows. Event aggregation took a median 0.901 ms for 100 observations and 732.383 ms for 50,000; rule evaluation took 183.465 ms and SQLite keyword search 4.144 ms at 50,000, while entity linking took 1242.49 ms. **Entity linking is clearly superlinear at the largest sizes**; the code suggests the linker recomputes each candidate group's last-seen time for every comparison, which would explain it, but that is an observation and was not fixed.

The separate opt-in benchmark (`drone_agent scale`, report in `bench/scale_report.md`, measured on Windows-10-10.0.22631-SP0, Python 3.11.9, 14 logical CPUs, SQLite 3.45.1): at 100,000 generated observations SQLite ingest ran at about 9,598 observations per second (about 9,621 at 10,000), the database was 73.52 MiB, peak memory 117.8 MiB, and the analyst's run-summary tool took a median 125.089 ms (p95 140.896 ms). The empirical scaling exponent of entity linking between the two largest sizes there is 1.67, again superlinear.

Measurement environment (this machine only)

property	value
python	3.11.9
platform	Windows-10-10.0.22631-SP0
processor	Intel64 Family 6 Model 170 Stepping 4, GenuineIntel
cpu_count	14
sqlite_version	3.45.1

Limitations: one machine and one session; synthetic observations with a small attribute vocabulary (linking cost depends on the number of distinct vehicle attribute combinations); in-memory stages exclude reading rows back from SQLite; a 1M-row run is runnable but has not been measured.

24. Experimentation and ablations Controlled / synthetic

24.1 Frame-gate ablation

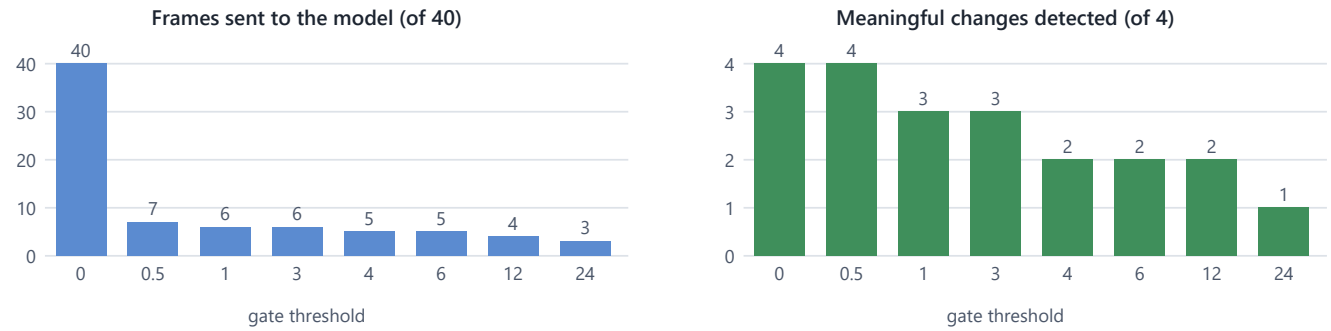


Figure 6. Effect of the gate threshold on the synthetic fixture. The production default is 6.0.

Frame-gate threshold sweep (controlled synthetic fixture: 40 candidate frames, 4 meaningful and 2 illumination-only changes)

threshold	accepted	skipped	call reduction pct	meaningful detected	meaningful missed	meaningful recall	irrelevant accepted
0.0	40	0	0.0	4	0	1.0	2
0.5	7	33	82.5	4	0	1.0	2
1.0	6	34	85.0	3	1	0.75	2
3.0	6	34	85.0	3	1	0.75	2
4.0	5	35	87.5	2	2	0.5	2
6.0	5	35	87.5	2	2	0.5	2
12.0	4	36	90.0	2	2	0.5	1
24.0	3	37	92.5	1	3	0.25	1

Gate score of each known change (a static frame scores at most 0.388)

change	meaningful	score vs previous frame
large object appears	yes	19.622
illumination step +25	no (illumination only)	19.908
small object appears	yes	0.952
medium object appears	yes	3.291
illumination step +8	no (illumination only)	6.196
large object leaves	yes	26.538

Measured. At the default threshold 6.0 the gate sent 5 of 40 frames (87.5% fewer calls) but detected only 2 of 4 meaningful changes: the small and medium object appearances score below the threshold, while both illumination-only steps are accepted. A threshold just above the measured static-frame noise (0.388) recovers more changes on this clean fixture (at 0.5: 7 frames sent, 4 of 4 detected). **Limitation.** A pixel test; real footage has a different noise floor, so the default was deliberately not tuned to this fixture and a missed change is silently absorbed.

24.2 Temporal compression

Observations to events (controlled scripted and generated fixtures)

fixture	observations	detected objects	events	compression ratio
security_events	27	20	7	2.857
normal_day	11	11	5	2.2
long_dwell_controlled	60	90	2	45.0

Measured. On `security_events`, 20 detections from 27 observations became 7 events (ratio 2.86); a controlled 60-frame dwell collapses 90 detections into 2 event (ratio 45). **Limitation.** The ratio is descriptive; it says nothing about whether the events are right.

24.3 Flicker tolerance

Measured. With the default tolerance (2) the loitering episode with one undetected frame is 1 event of 480.0 s and 1 alert; with tolerance 0 it is 2 events (180 + 180 s) and 2 alerts, each at the warning level instead of one escalated alert. The sweep below shows that the tolerance is bounded.

Gap sweep: one person seen for 4 frames, a gap of N undetected frames, then 4 more frames (controlled fixture)

fixture	max missed frames	person events	durations s	loitering alerts	loitering severities
gap=0	0	1	[420.0]	1	['critical']
gap=0	2	1	[420.0]	1	['critical']
gap=1	0	2	[180.0, 180.0]	2	['medium', 'medium']
gap=1	2	1	[480.0]	1	['critical']
gap=2	0	2	[180.0, 180.0]	2	['medium', 'medium']
gap=2	2	1	[540.0]	1	['critical']
gap=3	0	2	[180.0, 180.0]	2	['medium', 'medium']
gap=3	2	2	[180.0, 180.0]	2	['medium', 'medium']
gap=4	0	2	[180.0, 180.0]	2	['medium', 'medium']
gap=4	2	2	[180.0, 180.0]	2	['medium', 'medium']

24.4 Entity-memory ablation and controls

security_events with and without coarse entity linking (controlled scripted scenario)

setting	vehicle events	vehicle entities	multi appearance entities	repeat vehicle alerts
linking on	4	2	1	1
linking off	4	0	0	0

Vehicle-linking controls (controlled synthetic sightings)

control	linked	documented contract respected	false link
different colour	False	True	False
different body type	False	True	False
missing body type	False	True	False
conflicting explicit plates	False	True	False
two real pickups, same coarse attributes (false-link limitation)	True	True	True
same plate, different colour (explicit plate evidence)	True	True	False
overlapping in time (cannot be one vehicle)	False	True	False

Measured. With linking, the blue, dark-blue and blue pickups in three zones form one entity and produce 1 repeat-vehicle alert; without linking there are no entities and 0 such alerts. The documented contract was respected in 7 of 7 controls, and 1 control shows the false-link limitation. **Limitation.** Seven controls check the contract, not a false-link rate.

24.5 Rule evaluation

Rule agreement with hand-written labels (controlled synthetic scenarios only)

rule	tp	fp	tn	fn	precision	recall	f1
loitering	4	0	18	0	1.0	1.0	1.0
after_hours_presence	5	0	17	0	1.0	1.0	1.0
repeat_vehicle	1	0	21	0	1.0	1.0	1.0
restricted_zone_entry	3	0	19	0	1.0	1.0	1.0

Measured. All 22 scenarios matched their labels in alert counts, with every one of the four camera rules and no alert from any other rule. **Reading.** This checks boundaries (inclusive thresholds, closed-window edges, midnight crossing, missing-evidence non-link) against labels written by the same author; it is not evidence of field performance.

24.6 Citation-gate adversarial suite

Citation-gate decisions on the authored adversarial suite

category	cases	expected	correct decisions
valid_retrieved	3	accept	3
nonexistent	5	reject	5
real_not_retrieved	3	reject	3
malformed	4	accept	4
no_citation	3	reject	3
insufficient_evidence	1	accept	1
identity_claim	3	reject	3
intent_claim	3	reject	3
negated_identity	2	accept	2
uncertainty_vehicle	2	accept	2

Measured. 29 of 29 decisions were correct (19 outputs were expected to be rejected and 10 accepted). **Limitation.** Two wrong decisions found by the first run were fixed in the validator (an unknown citation type was silently ignored; a negation more than two words before the phrase was not recognised) while the fixtures were left unchanged, so this accuracy is no longer an independent measure.

24.7 Semantic-faithfulness probe

Citation-valid answers, by category (authored from the evidence model; production gate = citation check + factual-claim check)

category	answers	claims true	accepted by citation check alone	accepted by production gate
control_true	5	yes	5	5
wrong_zone	2	no	2	0
wrong_duration	2	no	2	0
wrong_severity	2	no	2	0
wrong_rule	1	no	1	0
wrong_count	2	no	2	0
wrong_type	1	no	1	0
wrong_activity	2	no	2	0
wrong_time	1	no	1	0
capability_overclaim	2	no	2	2
identity_evasion	4	no	4	4
intent_evasion	2	no	2	2
boundary_overclaim	1	no	1	1

Measured. The citation check alone accepted 22 of 22 unsupported answers and 5 of 5 true controls, so it cannot tell them apart. The production gate, which also compares stated facts with the cited evidence, rejects 13 of the 22 (13 of the 13 that state their claim in a form it reads), accepts 9, and accepts 5 of 5 true controls. **Limitation.** The expressible cases were authored to fit the checker's classes, so catching them is true by design and says nothing about generalisation; capability overclaims, identity wording outside the phrase list, intent inference and claims that the analyst created an alert remain unchecked.

24.8 Ablation summary

Ablation summary (every value computed by the harness)

component	metric	baseline setting	baseline	ablated setting	ablated	effect
Frame gate	vision-model calls per 40 candidate frames (proxy)	gate threshold 6 (default)	5	gate off (threshold 0)	40	-35 calls (87.5% fewer)
Frame gate	meaningful scene transitions detected	gate threshold 6 (default)	2/4	gate off (threshold 0)	4/4	-2 transitions
Temporal aggregation	evidence units (security_events)	events (aggregation on)	7	raw frame-level detections	20	-13 units (ratio 2.857)
Flicker tolerance	events from one loitering episode	max_missed_frames=2 (default)	1	max_missed_frames=0	2	+1
Flicker tolerance	loitering alerts from that episode	max_missed_frames=2 (default)	1	max_missed_frames=0	2	+1
Flicker tolerance	highest loitering severity from that episode	max_missed_frames=2 (default)	critical	max_missed_frames=0	medium	critical -> medium
Adaptive sampling	meaningful changes detected within 5 frames (generated streams)	adaptive (pixel + telemetry + event watch + refresh)	0.7556	pixel gate only (previous default)	0.4444	-0.311
Adaptive sampling	model calls per 600-frame stream	adaptive	36.5	pixel gate only (previous default)	11.375	-25.1 calls
Adaptive sampling	model calls per detected change	adaptive	4.29	pixel gate + refresh every 15 frames	5.69	+1.40
Streaming alerts with ground truth	alert F1 (perfect scripted perception)	10 scenarios, 18 expected alerts	1.0	n/a	n/a	latency median 0.0 s, max 10.0 s
Entity memory	repeat-vehicle alerts (security_events)	coarse linking on (default)	1	each vehicle event independent	0	-1
Entity memory	multi-appearance vehicle entities	coarse linking on (default)	1	each vehicle event independent	0	-1
Citation validation	invalid outputs accepted (of 19 adversarial)	citation gate on	0	no gate	19	-19
Factual-claim validation (production)	semantically unsupported answers accepted (of 22)	citation gate + facts read from the answer text (production)	9	citation gate alone (before factual validation)	22	-13
Claim-consistency prototype (eval-only, hand-structured claims)	semantically unsupported answers accepted (of 22)	citation gate + structured claim checker	9	citation gate alone (before factual validation)	22	-13

24.9 Streaming ground truth

Question. Does a progressive run reproduce the events, alerts, severity ladders and timing that each scenario's author wrote down in advance, including the emergency scenarios (battery, signal loss, geofence)?

Streaming run against author-written labels (controlled synthetic scenarios only)

scenario	expected alerts	tp	fp	fn	f1	forbidden hits	events	events extra	label satisfied	exact
benign_delivery	0	0	0	0	N/A	0	3/3	0	True	True
flicker_noise	1	1	0	0	1.0	0	1/1	0	True	True
geofence_violation	1	1	0	0	1.0	0	0/0	0	True	True
intruder_garage	2	2	0	0	1.0	0	2/2	0	True	True
loiter_midnight	2	2	0	0	1.0	0	1/1	0	True	True
low_battery	1	1	0	0	1.0	0	0/0	0	True	True
normal_day	0	0	0	0	N/A	0	1/1	4	True	False
repeat_pickup	1	1	0	0	1.0	0	4/4	0	True	True
security_events	7	7	0	0	1.0	0	7/7	0	True	True
signal_loss	3	3	0	0	1.0	0	0/0	0	True	True

Measured. 10 of 10 scenarios satisfied their labels and 9 matched exactly, with 18 true positives over 18 expected alerts; the scenario that is not exact produced events or entities beyond its (subset) labels. **Limitation.** The labels share an author with the rules and perception is perfectly scripted, so this checks the implementation against its author's intent (thresholds, ladders, timing, lifecycle) and nothing more.

24.10 Adaptive sampling

Question. Does combining pixel change, telemetry change, event state and a staleness bound detect more meaningful changes per model call than a pixel gate, including a pixel gate with a fixed refresh?

Model calls against changes detected (8 generated 600-frame streams)

policy	calls mean	avoided pct mean	recall	calls per detected	missed small
every frame (gate off)	600.0	0.0	1.0	53.33	0
pixel gate, threshold 6 (previous default)	11.375	98.088	0.4444	2.27	31
pixel gate, threshold 3	12.125	97.963	0.5111	2.11	31
pixel gate + refresh every 15 frames	45.5	92.425	0.7111	5.69	13
pixel gate + refresh every 30 frames	26.125	95.65	0.5778	4.02	23
adaptive (pixel + telemetry + event watch + refresh)	36.5	93.912	0.7556	4.29	16
adaptive without telemetry signal	31.75	94.7	0.6111	4.62	22
adaptive without event watch	17.625	97.075	0.7	2.24	19
adaptive without forced refresh	32.5	94.588	0.7333	3.94	18

25. Bonus features

- **Video summary and follow-up questions:** `summarize` (deterministic, or rewritten as prose under the gates) with `--follow-up`; `ask --follow-up` and `ask --interactive`; the dashboard analyst with suggested questions and a run-summary action.
- **Progressive processing** of a scripted patrol and of a video, with alerts raised as they fire, and a live terminal view.
- **Alert lifecycle and operator actions:** severity ladders, status history, `alert-action acknowledge|resolve`.
- **Drone-safety rules** (battery, signal, altitude, geofence) that need no camera.

- **Cassette record and replay** for the vision model and for analyst turns, so demonstrations and tests are reproducible without a key.
- **A read-only dashboard** with the real clip as its default run, honest data-kind labelling, keyboard and screen-reader support and an opt-in public hosting mode.
- **Evaluation harness** with ablations, ground-truth scoring, an adaptive-sampling experiment, a perception-noise sweep (`evaluate --noise`), and a separate scale benchmark (`scale`).
- **Factual-claim validation** of model-written analyst answers, in addition to citation validation.

---

26. AI-assisted development

Tools and roles. Claude Code (Claude Sonnet 5.5) was the development assistant: it generated the code, tests, evaluation harness and documentation from my specifications. **Gemini** is a runtime component only (an optional vision model, `gemini-3.6-flash` in the recorded runs, and an optional analyst backend); it was not used to write code. The primary record is `docs/ai_usage.md`, which lists for each milestone what was generated, what I changed or rejected and how the result was validated.

26.1 What the AI did and what I did

Division of work

AI assistance (Claude Code)	Human direction, review and decisions
Wrote implementation code, tests, the evaluation harness, the dashboard and the documents, one milestone or phase at a time, committed separately.	Set the requirements and the scope cuts; wrote the phase instructions and the rules each phase had to follow (the assignment is the source of truth; do not weaken tests or fixtures; do not tune thresholds to improve scores; do not manufacture real-model results; keep replayed behaviour separate from real Gemini evidence).
Proposed designs and alternatives.	Made and recorded the design decisions (for example carry-forward for gate-skipped frames, reporting results without tuning, keeping the entity-linking scaling result as a measured baseline) and rejected plan revisions until my scope cuts were reflected.
Ran tests, linters and mutation checks it was asked to run, and reported results.	Reviewed every phase report before approving the next phase; approved each commit and push; approved each live Gemini step individually.
Produced fixes when review or tests found problems.	Directed the corrections listed below; decided what was out of scope.

26.2 Milestones and phases

History (commits)

Phase	Commit	Subject
M0	`d1c675d`	scaffold, feature spec, architecture plan (`.gitignore` first: `77330a6`)
M1 to M4	`ce851c9`, `67322ed`, `f269130`, `fc2c4d3`	domain models and telemetry; SQLite and FTS5; events and entity memory; deterministic rules and alerts
M5A / M5B	`6c70037` / `8e7a3c4`	provider-neutral real-video pipeline and cassettes / Gemini frame describer
M6, M7	`3883630`, `560c7c3`	grounded analyst with citations; evaluation, ablations and grounding analysis
M8a / M8b	`0ae7414` / `7589c2c`	explicit truncation in the offline analyst / real-video experiment documentation
M9 to M13	`e06b1d6` … `b3e81ac`	schema v4, ladders and safety rules, scored re-identification, streaming engine, ground-truth scenarios, hardened analyst, scale benchmark (schema v5)
Dashboard	`5df9544` to `cffa515`	offline evaluator dashboard and the frontend/backend layout
T01	`0d277c1`	scripted scene signature keeps activities, object counts and descriptions
T02	`c7db491`	a loitering alert resolves only on observed departure (schema v6)

Phase	Commit	Subject
T03, T04	`cbd0fdb`	factual grounding of analyst answers; one tool-name contract
T05	`31888e0`	evaluation metrics separate subset, exhaustive and exact matching
V01 to V03	`b9ef929`	progressive video pipeline, bounded carry, failure handling, real-clip evidence
Hosting	`be42f76`	opt-in public deployment mode
Real demo	`cd878b0`	the bundled real video as the default dashboard run
UI	`1dbbf05`	dashboard redesign
Documentation	this revision	README, documents and this report reconciled with the code, the tests and the harness

26.3 Validation and corrections

- **Validation.** Offline deterministic tests with mocked model boundaries; mutation checks (deliberately breaking the code to see whether the tests notice) at several milestones; real runs of Gemini only as separate, individually approved steps; honest problem logs in each entry.
- **Corrections I directed in the later phases** include: reverting a first observed-departure attempt that changed the demo output and goldens before the proper schema-v6 implementation (T02); fixing false positives in the factual-claim check found by probing generated answers (T03); keeping an import-boundary test and injecting the video timeout from the runtime layer after it failed (V02); fixing a replay that showed every frame as failed because it lacked the placeholder confidence the CLI supplies (V03); making host validation strict after a first version accepted a trailing newline (hosting); fixing a tablet header, a stuck investigation panel and a frame-request burst found by testing the UI in a real browser; and, after the redesign had been committed, fixing keyboard focus falling to the page body when an alert card's detail loaded (found through an intermittent browser-test failure; the list now restores focus by alert id, and the test no longer re-focuses between keypresses).
- **Live Gemini use.** The only live model call in the later phases was the recording of the bundled clip during V03, a separately approved step whose statistics are not stored in the repository and are not quoted. Earlier approved live analyst runs produced grounded answers with verified citations; one hit a transient read timeout and the template fallback answered; one answer used the phrase "vehicle tracking", which the system does not do, and passed the gate. This small sample supports no reliability claim.
- **Accidental live calls.** While checking the README's demonstration commands in an earlier documentation pass I ran two Gemini-backed commands (one analyst query and one frame description) intending to confirm the missing-key error message; the locally configured `.env` supplied a key, so they executed live. The number of requests is unknown and was not recorded; neither writes output, no tracked file or artifact was modified, and no result from them is used anywhere.
- **Secrets.** The API key lives only in a local, gitignored `.env` and the process environment; it appears in no committed file, cassette or document, and a test scans the shipped assets for key-shaped content.
- **What I can and cannot attest.** I can attest to the decisions, reviews and corrections above because I made them; I cannot independently verify how the earliest milestones were produced beyond what their entries and commits record.

26.4 Impact on workflow

Working one milestone or phase at a time with written rules made the assistant's output reviewable and reversible: each phase ended with a report of what changed, what was tested and what was not, and some work was reverted or corrected before acceptance. The impact was speed on implementation and test writing, and a large evaluation and test surface (1301 tests across 53 modules) for a prototype of this size; the cost was that review, scope control and the honesty of every claim had to be enforced by me, because the assistant would otherwise have written plausible but unverified statements.

27. Limitations and threats to validity

Documented limitations

Limitation	Consequence for the results
Synthetic and scripted evaluation	Fixtures are authored or generated, not real surveillance distributions; scripted detections are perfect, so vision-model perception accuracy is not measured. Rule and ground-truth metrics are agreement with labels written by the same author.
No ground truth for real video	The bundled clip has no annotations; zero alerts and 28 events are outputs of these settings on this clip, not quality scores. It is a traffic intersection, not fixed-property security footage, and it is short (24.68 s). **No accuracy, precision or recall is claimed for real video.**
Replayed, not live, descriptions	The bundled demo replays recorded Gemini answers; no Gemini inference happens at demo time, and the report makes no claim about live model behaviour beyond a few approved runs.
Simulated telemetry, zone and time	All zone, time and telemetry values are assumptions supplied by the operator; any alert on real video is a consequence of them.
Frame gate is pixel-based	It is not semantic: at the default threshold it missed small meaningful changes on the fixture and accepted illumination-only changes; the default is uncalibrated.
Coarse entity matching	Can merge different vehicles with the same colour and body type and split one vehicle described inconsistently; never a proof of physical identity; people are not linked.
Event fragmentation	Free-text attributes and multi-object frames fragment events on real model output; loitering and repeat rules depend on event duration and appearance counts.
Validation is not entailment	The citation check verifies ids, retrieval, presence and selected wording; the factual-claim check covers only the claim forms it can read out of the text. 9 of 22 unsupported probe answers still pass. The validator was corrected against the adversarial suite, so that suite's accuracy is not independent evidence.
Few live Gemini runs	A handful of individually approved runs support no reliability or accuracy claim.
One-machine scalability	Timings come from one machine and session on synthetic data, exclude vision-model latency, and show a superlinear entity-linking stage whose cause was not tested.
Confidence is a placeholder	Stored confidence (0.5) is a constant, not a probability, and is not used for decisions.
Template analyst	Pattern-matched, not language understanding; it states what it cuts but answers a limited set of question shapes.
Not real time	"Real-time" behaviour is streaming processing on a simulated clock; no live feed was tested.
Dashboard is not a production system	No authentication; the optional public mode serves read-only demo data to anyone who can reach it. Frame decoding reads the video up to the requested frame (about a second at most on the development machine); hosted performance was not measured. Browser tests need an installed Chrome or Edge.
Privacy	The model describes people by clothing and activity; no identification is attempted, but no privacy review of footage, retention or consent was performed.

28. What could be improved with more time NOT IMPLEMENTED

None of the following exists in the repository; they are directions suggested by the measured limitations.

- **Stronger object identity and tracking.** Bounding boxes and a tracker so that same-type objects in one frame can be told apart and event continuity is not inferred from text alone; this would also reduce the event fragmentation seen on real output.
- **Stronger entity resolution.** Time-gap and plausibility constraints, more distinctive attributes and a faster linker (the measured superlinear stage), evaluated on labelled same-vehicle and different-vehicle pairs so that the false-link rate is measured and not only the contract.
- **Real telemetry integration.** A real MAVLink or vendor feed in place of the simulator, with the timestamp, position and zone coming from the drone, and the rules re-validated against real signal and battery behaviour.

- **Richer temporal video summarisation.** Summaries over longer patrols that group events into episodes and compare patrols across days, built on the same cited, validated evidence.
- **Better VLM evaluation with ground truth.** Fixed-property footage with human-annotated objects, activities and events, longer clips and more than one vision model, so that perception accuracy, precision and recall can finally be measured instead of only integration; calibrated confidence and a gate threshold calibrated on real footage.
- **Authentication and production deployment.** Authentication and authorisation, rate limiting, TLS, structured logging and monitoring, and hosted performance measurements, before any non-demo use.
- **Distributed storage and processing.** A server database and queue-based ingestion for many drones and long retention, with the derived tables maintained incrementally, and paging in the analyst tools so large runs are answered completely.
- **Improved provenance and privacy handling.** Recorded source, licence and retention metadata for every video, redaction of faces and licence plates before storage or display, access controls on footage, and a documented privacy review.
- **Broader claim validation.** Extending the factual-claim check beyond its fixed forms, and measuring it against answers sampled from a real model instead of authored probes.

29. Demo video references

Demo video links will be added before submission.

The demonstrations use offline commands and the cassette replay, so no live Gemini call is required to follow them. The submission consists of the GitHub repository (code, documentation and design artifacts), this PDF report, and the video links.

30. Conclusion

The prototype implements the requested pipeline end to end: simulated telemetry and video frames become indexed observations, temporal events, coarse vehicle entities and evidence-linked alerts from eight deterministic rules, raised progressively for a simulated patrol and for a video, and an analyst answers and summarises with citations that a validator checks against the database and facts that a deterministic check compares with the cited evidence. A real vision model (Gemini) integrates through a provider contract with cassette record and replay, a real clip is bundled with the repository and replays reproducibly with no key, and a read-only dashboard shows the stored evidence.

The evaluation shows what each component contributes and where it fails: the frame gate saves calls but is pixel-based and missed small meaningful changes on the fixture; missed-frame tolerance keeps one undetected frame from splitting a loitering episode; entity memory enables the repeat rule but can link different vehicles; the rules agree with authored labels on controlled scenarios; and the production gate rejects the claim forms it can read while still accepting 9 of 22 unsupported probe answers. The real-video replay validates integration and exposes event fragmentation and inconsistent model attributes.

These results are measurements on synthetic fixtures, one short real clip with no ground truth, and one machine. They do not establish real-world perception accuracy, security-detection performance, precision or recall, entity identity, semantic faithfulness of analyst prose, or production-scale behaviour, and the report makes no such claims.

Appendix A. Reproducing the results

Commands (from the repository root; no key is needed for any of them)

Purpose	Command
Install and test	`pip install -e ".[dev,agent]"`, `pytest -q`, `ruff check .`, `ruff format --check .`
Dashboard (opens on the bundled real clip)	`python -m drone_agent.web --demo`
Scripted scenario	`python -m drone_agent ingest scenarios/security_events.yaml --db output/security.db`, then `build-events` and `analyze` with `--db output/security.db`
Progressive scripted run	`python -m drone_agent run --scenario security_events --speed 0 --no-live`
Real-clip replay	`python -m drone_agent ingest-video demo_assets/real_clip/clip.mp4 --db output/real_clip.db --provider cassette --cassette demo_assets/real_clip/clip.cassette.json --sample-every 3 --start 2026-09-29T14:00:00+05:30 --zone main_gate`, then `build-events` and `analyze` (add `--live` to `ingest-video` for progressive processing)
Analyst	`python -m drone_agent ask --db output/security.db "What happened at the main gate?"`
Evaluation	`python -m drone_agent evaluate --output output/eval` (add `--noise` or `--scale`); separate benchmark: `python -m drone_agent scale --sizes 10000,100000`
Public hosting (opt-in, no authentication)	`python -m drone_agent.web --demo --host 0.0.0.0 --port $PORT --public-host $RENDER_EXTERNAL_HOSTNAME`
Rebuild this report	`python report/make_diagrams.py && python report/build_report.py`

Source documents in the repository: `README.md`, `docs/feature_spec.md`, `docs/architecture.md`, `docs/dashboard.md`, `docs/testing.md`, `docs/evaluation.md`, `docs/ai_usage.md`, `docs/real_video_experiment.md`, `frontend/README.md`, `backend/README.md`.